

Ziplo — Project Status Report & Next Stage Roadmap

Status Date: July 17, 2026 | Environment: Development / Monorepo

1. Executive Summary

Ziplo is a localized e-commerce and delivery tracking platform connecting regional shops (merchants) with customers. The project features a fast and clean FastAPI backend backed by an ORM (SQLAlchemy) layer, and a highly responsive frontend SPA. Recently, the codebase was audited for common vulnerabilities, optimized by removing redundant legacy routes/schemas, and prepared for next-stage deployment and production security hardening under the new legal entity branding.

2. Current Architectural State

Backend (FastAPI):

- **Database:** SQLite database (gokirana.db) dynamically managed via SQLAlchemy ORM.
- **File Storage:** S3-compatible cloud storage via boto3 Integration for product and profile images.
- **WebSocket Gateway:** Dedicated websocket connection manager routing live notifications (e.g. order updates) dynamically.
- **Authentication:** Password-based login combined with email/WhatsApp OTP verification simulator (OTP codes logged in server output).

Frontend (React):

- **Customer Portal:** Browsing products, adding items to cart, selecting locations, and tracking order updates dynamically.
- **Merchant Hub:** Inventory creation, tracking incoming WebSocket order signals, updating delivery tracking coordinates, and updating statuses.
- **Command Center:** Admin metrics summarizing shops, orders, total sales, and order success rates.

3. Optimizations & Security Improvements Completed

- **Long Password DoS (LPDoS) Mitigated:** Implemented explicit password character limit restrictions (max_length=128) across active authentication schemas using Pydantic Field validator capabilities.
- **Redundant Code Pruned:** Deleted dead legacy endpoints (/api/shops/register, /api/shops/login), unused models/schemas (ShopCreate, ShopLogin), and dead websocket messaging functions. This resolved an un-validated password input pathway.
- **Database Truncated:** Successfully reset and initialized database state for clean onboarding trials.
- **S3 Storage Cleaned:** Fully cleared stale/orphaned media assets from S3 buckets.

4. Next Stage Requirements & Roadmap

To advance Ziplo to a production-ready state, the following critical requirements must be implemented:

Category	Requirement Description	Priority
Security	JWT Authentication: Implement secure JSON Web Token workflow to protect backend endpoints.	Critical
Security	Password Hashing: Incorporate bcrypt or argon2 to hash passwords securely rather than storing plaintext in SQLite.	Critical
Infrastructure	Secrets Management: Rotate S3 credentials and transition active environment variables to secure vault storage.	High
Core Logic	Rate Limiting: Put rate limiters in place to block brute-force attempts on login and verify routes.	High
Notification	Live OTP Gateway: Link real-world SMS/Email providers (e.g. Twilio/SendGrid) for customer notifications.	Medium
Operations	HTTPS/WSS Enforcement: Require transport-layer security across all environments.	Medium